SAP BTP Pro-Code

Cloud Application Programming (CAP)

Version: 1.0
Date: 2025-10-22
Owner: Maximilian Eckert

Overview and Important Notes

IMPORTANT		
BTP Application User	**[AppUser]**	:
Individual ID	**[###]**	:
Institutional ID	**[$$$]**	:
Business Application Studio	**[BAS]**	:
Origin Key Identity Provider	**[KeyIP]**	:
Cloud Foundry Endpoint	**[CFEnd]**	:
Cloud Foundry Organization	**[CFOrg]**	:
Cloud Foundry Space	**[CFSpace]**	:
CAP Project	**[Proj]**	:

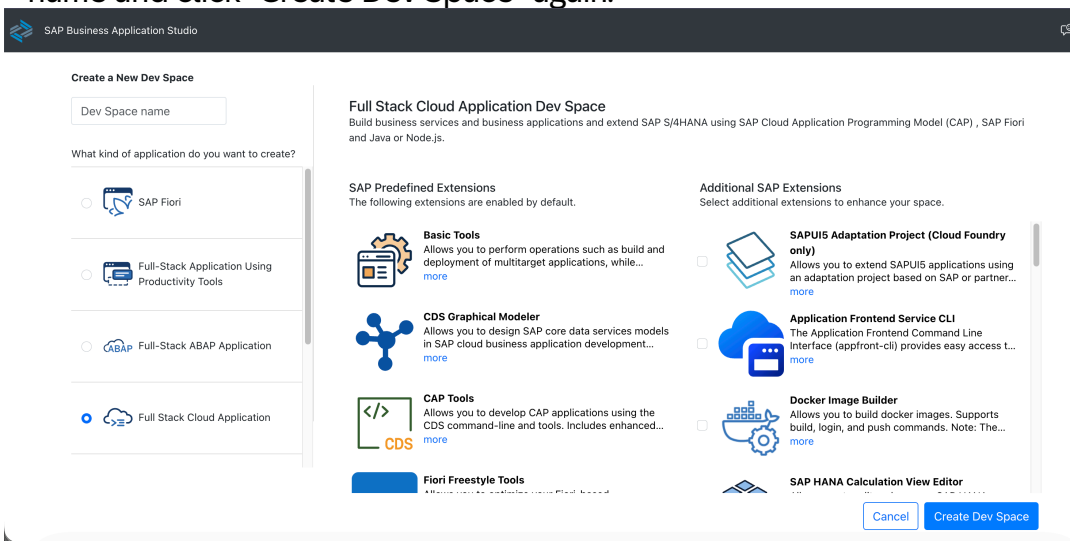
Note: Every time you encounter the tags **[###]**, **[$$$]**, ... you need to use the corresponding value from the table above.

Please decide: either use Business Application Studio or develop locally in your VS Code installation.

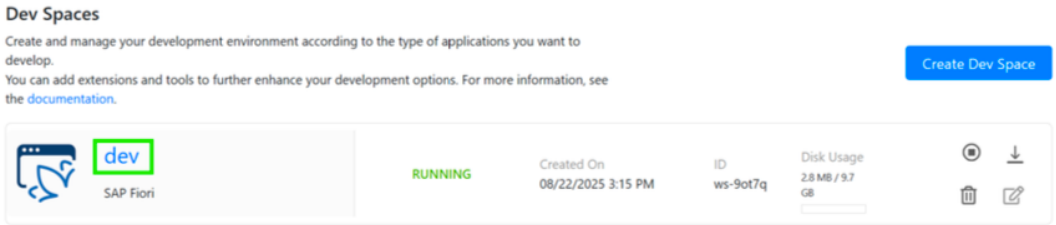
1. Setup Preperation

Option A: Create a Dev Space in Business Application Studio

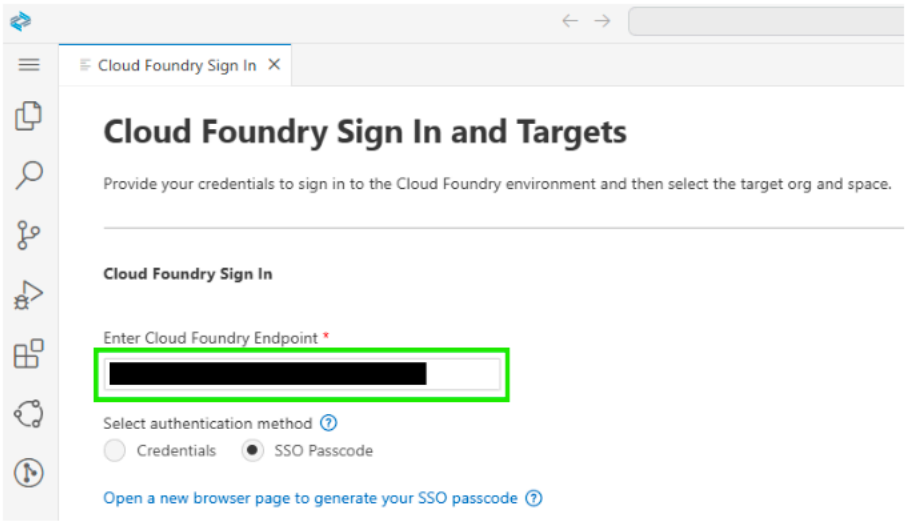
1. Open the Business Application Studio (BAS) Link **[BAS]** provided by your instructor and login with your **[AppUser]** provided by your instructor.
2. Then press “Create Dev Space”, choose “Full Stack Cloud Application”, assign a name and click “Create Dev Space” again.



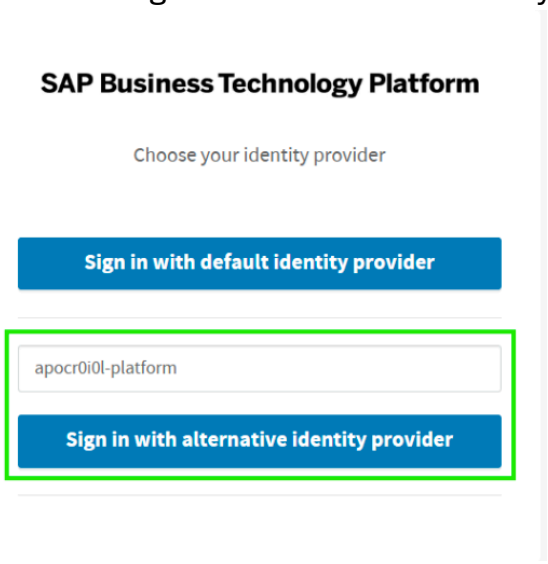
3. To deploy an application to the Cloud Foundry Space, you need to logon to Cloud Foundry. For that, enter your newly created dev space. Wait until the status is “Running” then click on the name of the dev space.



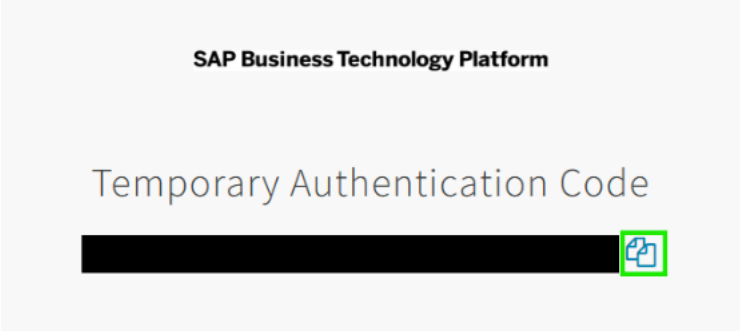
4. In the menu click **View->Command Palette (Ctrl + Shift + P)** and start typing CF: to see **CF: Login to Could Foundry** in the dropdown. Click this entry.
5. In the next window make sure that your CF Endpoint is correct comparing to **[CFEnd]**. Select “SSO Passcode” as authentication method. Click “Open a new browser page to generate your SSO passcode”.



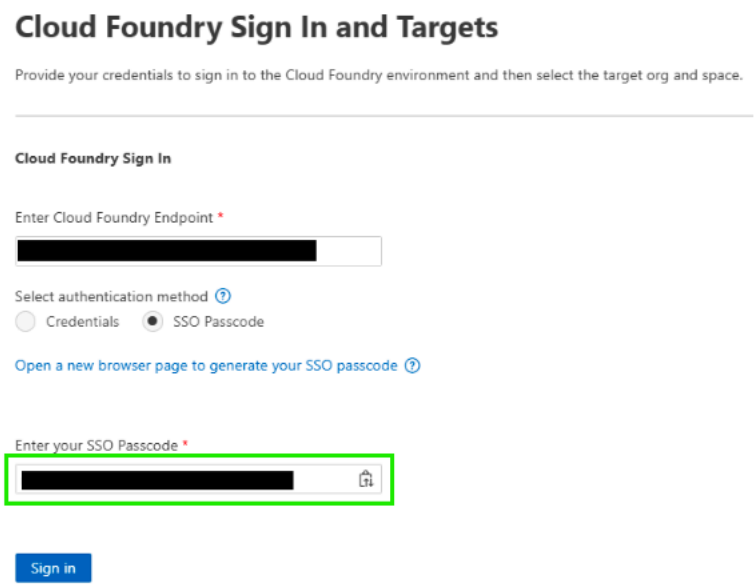
6. A browser page will open. Enter the origin key of the identity provider **[KeyIP]** and click “Sign in with alternative identity povider”



7. In the following login screen enter **[AppUser]** and the password provided by your instructor. Copy the SSO Code to the clipboard.



8. Go back to BAS, enter the SSO Code and click “Sign in”.



9. Make sure the “Selected Cloud Foundry Organization” and “Selected Cloud Foundry Space” are correct comparing to **[CFOrg]** and **[CFSpace]**. Click Apply.

10. Congratulations! 🥳 - You are now connected to your Cloud Foundry Space.

Option B: Setup your local VS Code instance

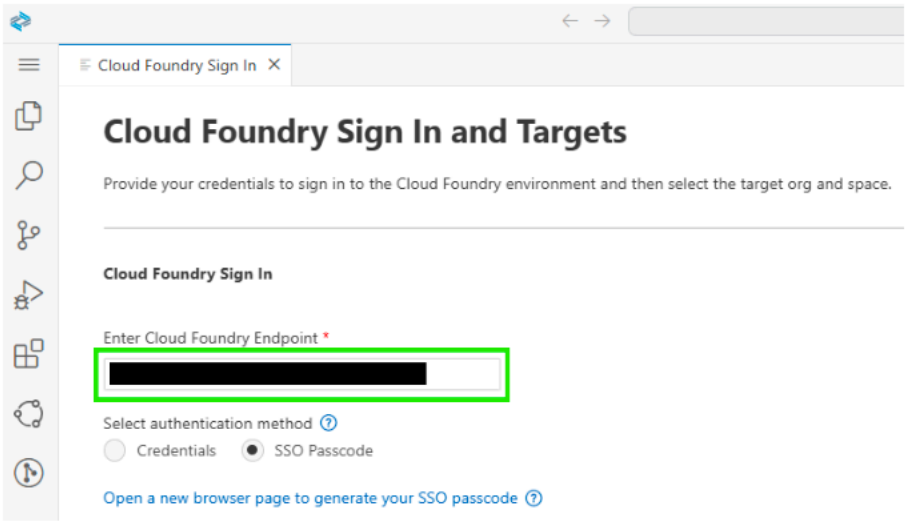
1. Make sure VS Code including Node.js version 22 (or 22) is installed on your computer.
2. Install the following dependencies globally:

```
# Install cds toolkit
npm add -g @sap/cds-dk

# Install cloud-mta-build-tool to build a multi-target cloud
application
npm add -g mbt

# Install UI5 cli
npm install -g ui5/cli
```

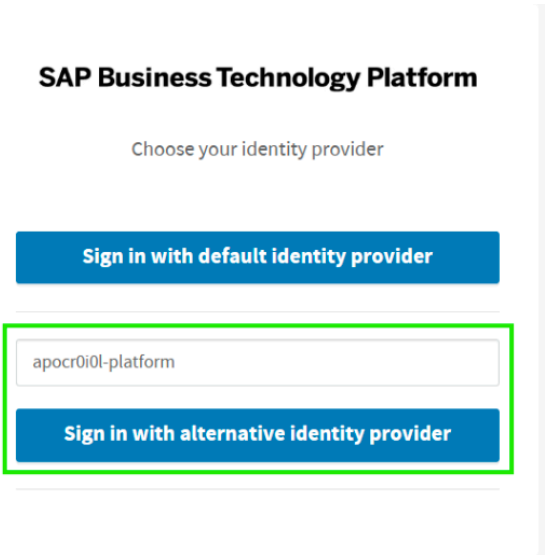
3. Install CloudFoundry cli
4. Install necessary CloudFoundry plugins:

```
# Add CloudFoundry community repo
cf add-plugin-repo CF-Community
https://plugins.cloudfoundry.org

# Install CloudFoundry multiapps plugin
cf install-plugin -f multiapps

# Install CloudFoundry HTML5 plugin
cf install-plugin -f html5-plugin
```

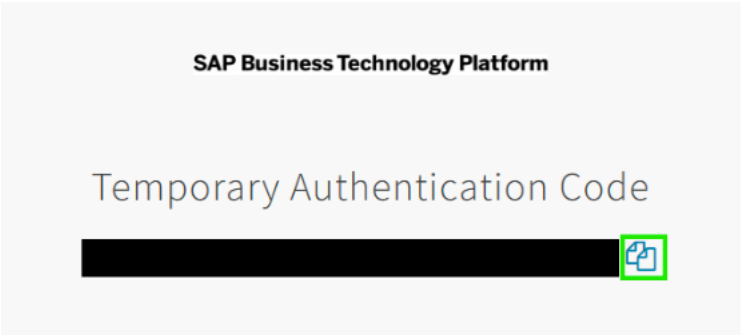
5. Install the following extensions in VS Code:
   - **Cloud Foundry Tools**
   - **SAP Fiori Tools - Extension Pack**
   - **SAP CDS Language Support**
   - **Rainbow CSV**
   - Optional: **core data services graphical modeler for VS Code**
6. In the VS Code menu click **View->Command Palette (Ctrl + Shift + P)** and start typing CF: to see **CF: Login to Could Foundry** in the dropdown. Click this entry.
7. In the next window make sure that your CF Endpoint is correct comparing to **[CFEnd]**. Select “SSO Passcode” as authentication method. Click “Open a new browser page to generate your SSO passcode”.



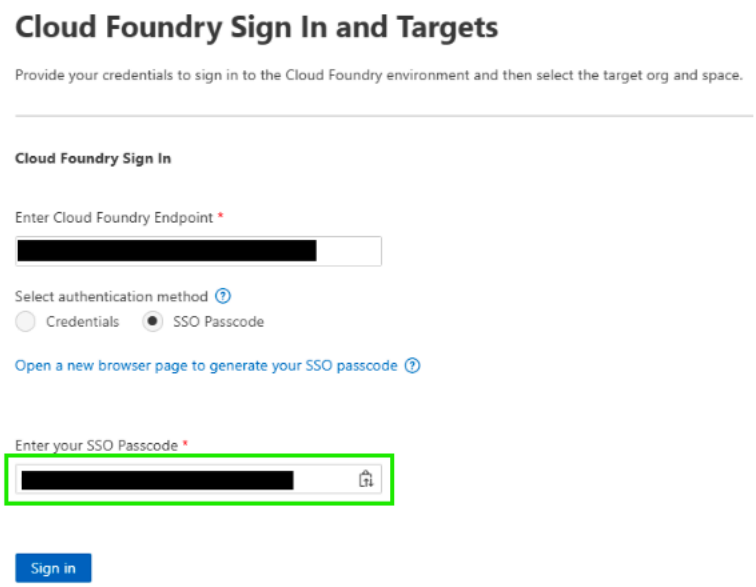
8. A browser page will open. Enter the origin key of the identity provider **[KeyIP]** and click “Sign in with alternative identity provider”



9. In the following login screen enter **[AppUser]** and the password provided by your instructor. Copy the SSO Code to the clipboard.



10. Go back to VS Code, enter the SSO Code and click “Sign in”.



11. Make sure the “Selected Cloud Foundry Organization” and “Selected Cloud Foundry Space” are correct comparing to **[CFOrg]** and **[CFSpace]**. Click Apply.

12. Congratulations! 🥳🎉 - You are now connected to your Cloud Foundry Space and successfully setup your local development workspace.

2. Task description

Develop a comprehensive full-stack university management application using SAP Cloud Application Programming Model (CAP) that serves as a centralized system for managing academic operations. The application will handle the organization of academic programs (studies), course modules, and student enrollment in a structured and efficient manner.

This enterprise-grade solution will provide:

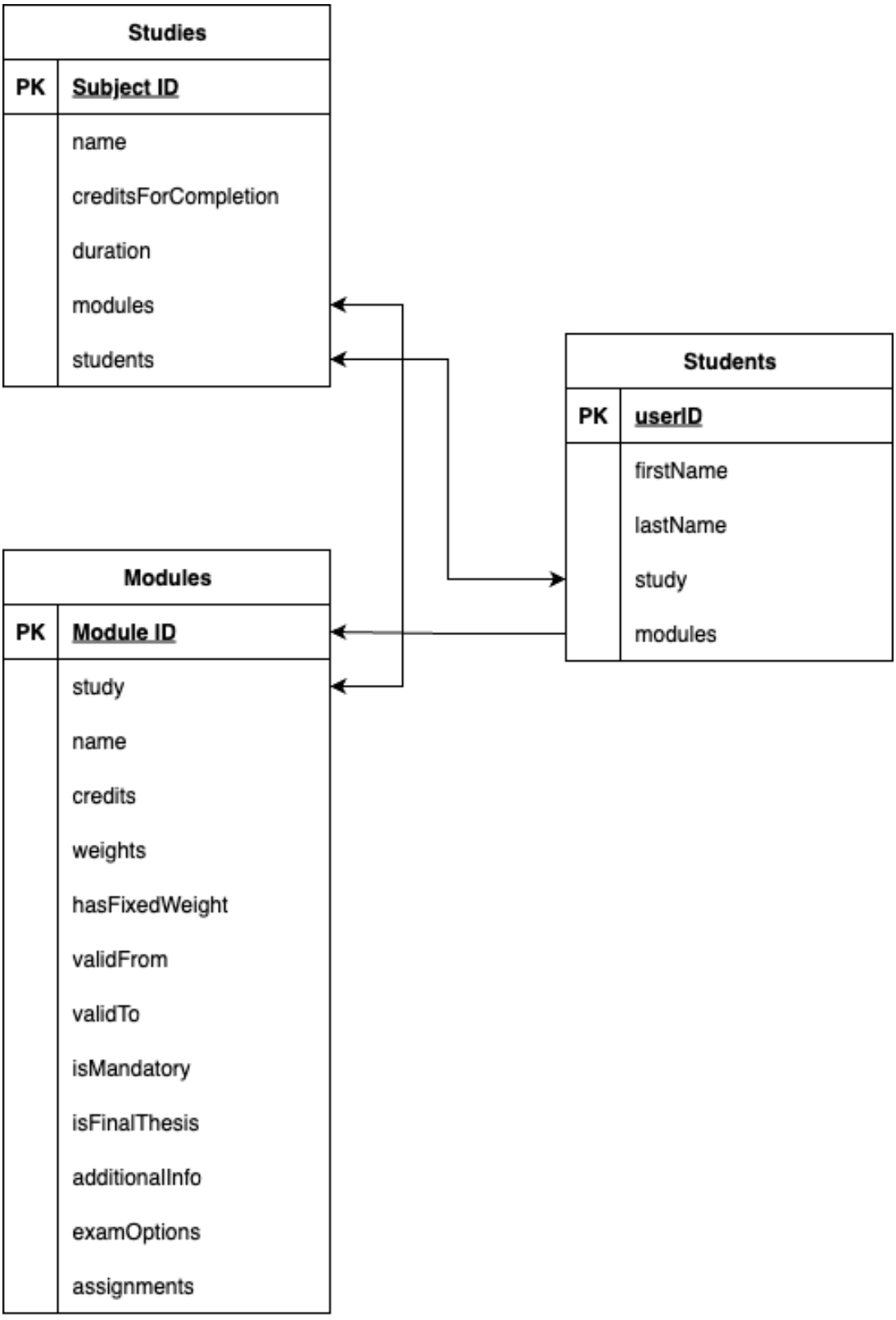
- **Study Program Management**: Organize and maintain different study programs (degrees) with their associated requirements including credits needed for completion, program duration, and curriculum structure
- **Module Catalog Management**: Define and manage course modules offered across study programs, including credit values, weights, validity periods, mandatory/elective status, exam options, and assignment tracking
- **Student Management**: Track students throughout their academic journey, managing their enrollment in study programs and registration for specific modules, maintaining comprehensive student records including personal information and academic progress
- **Module Assignment Tracking**: Enable students to register for available modules and manage their course selections, while enforcing business rules around module availability, prerequisites, and study program requirements

The system establishes clear relationships between Students, Studies (degree programs), and Modules, allowing for flexible curriculum design while maintaining data integrity and providing students with a personalized view of their academic path.

Helpful links

- Fiori development portal & Fiori elements documentation
- CAP documentation
- SAP Learning

3. Data Model

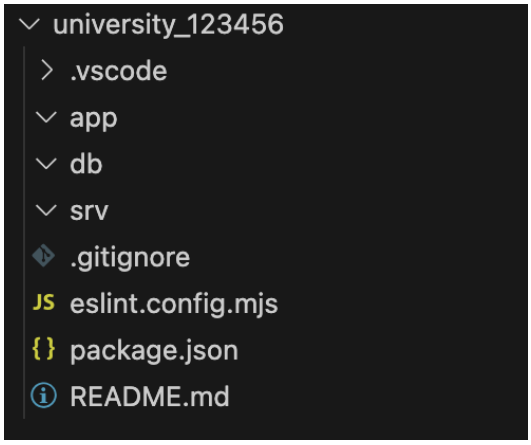

Studies
PK
Subject ID
name
creditsForCompletion
duration
modules
students
Students
PK
userID
firstName
lastName
study
modules
Modules
PK
Module ID
study
name
credits
weights
hasFixedWeight
validFrom
validTo
isMandatory
isFinalThesis
additionalInfo
examOptions
assignments

4. Getting started

1. Initalize your CAP application with the following command in the terminal either in VS Code or BAS

```
cds init university_$$$###
cd university_$$$###
```

2. Your project structure should look like this:



3. File Structure Setup:

   Create database schema files in `/db/`:

   - students.cds
   - studies.cds

   Create a data directory at `/db/data/` with CSV files:

   - sap.cap.sample-Students.csv

   Create service definition files in `/srv/`:

   - students-service.cds
   - studies-service.cds
   - catalogue-service.cds

4. Create example data by running the following command:

```
# Generate only schema
cds add data
# Generate schema with example data
cds add data --records 10
```

5. Create the data models based on the entity diagram in the /db/ folder. You can have a look at the documentation for reference

   Hint: if you are stuck with the creation of the data model, please raise your hand. We are happy to help you. ✋

6. Test if your data models are defined correctly and no compilation errors occur (You can leave this command running while working on the next steps):

```
cds watch
```

7. Annotate labels in the /db/ folder by annotating "@title", to all properties. Furthermore annotate "@UI.Hidden" to the properties, which should not be visible on the UI.
   - (Optional) If you want to localize all labels, you can write as the string:

```
@title: '{i18n>MY_I18N_KEY}
```

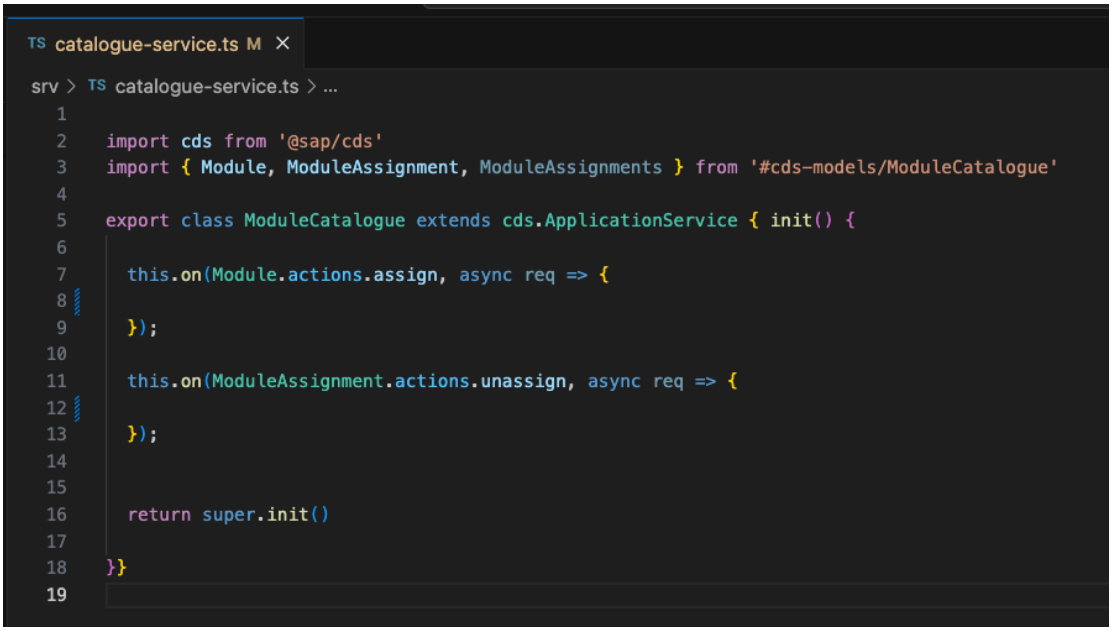
8. Create the services in the /srv/ folder, which expose the entities via API.
   - Check out the documentation for how to define services.
   - The students service should expose the **Students** entity and the **Studies** entity. The studies entity can be later used as a value help when creating a new student and assigning them to a study.
   - In the Students and Studies service enable draft for the respective entities to allow them being edited.
   - In the Catalogue service expose the **Modules** entity as well as the **ModuleAssignments** entity, to show a list of modules students can sign up for and the list of modules the logged in user is assigned to.
     i. Actions allow to define POST endpoints, which execute some logic when being called. Follow the docs to add a bound action "assign" to the **Modules** entity and a bound action "unassign" to the **ModuleAssignments** entity.
     ii. With @restrict you can define access restrictions for exposed entities. Define an action restriction for the 'READ' event for the **ModuleAssignments** entity, so that only those module assignments can be read where the user Id equals the logged in user id (which can be accessed via the pseudo variable $user.id).

iii. In addition, create a READ restriction for **Modules**, that logged in **Students** can only see modules, for the study they are assigned to.
iv. Furthermore, add restrictions for both bound actions, that "assign" can only be executed when the student is not yet assigned and "unassign" when the user owns the module assignment.

9. Implement the service handlers for both "assign" and "unassign" actions
   - Run the following commands to add the TypeScript configuration:

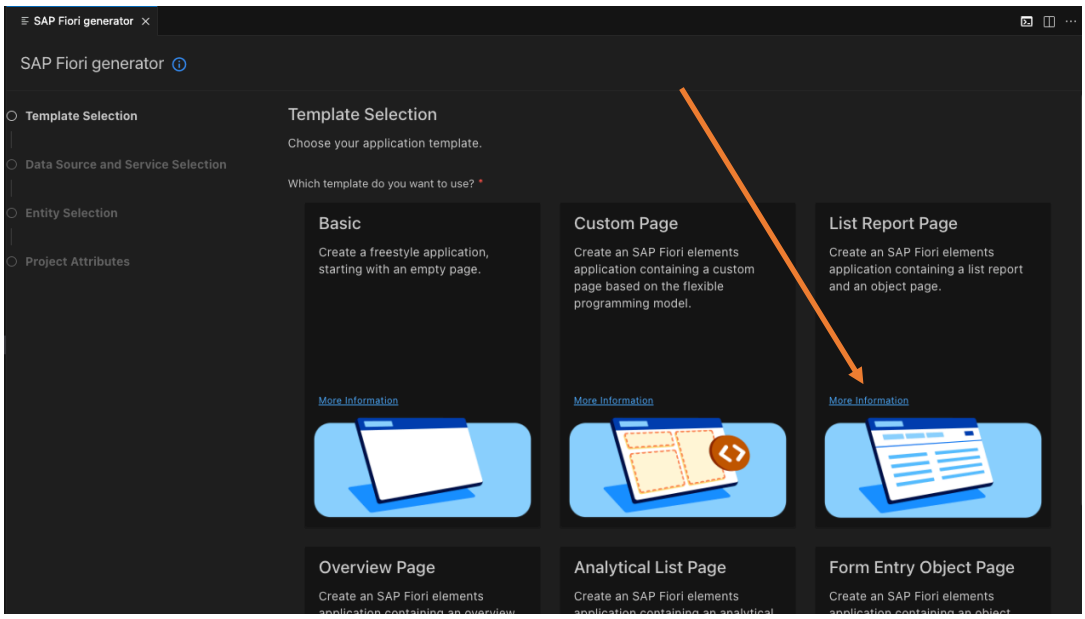
```
cds add typescript
npm install
```

   - Add a `/srv/catalogue-service.ts` file. CAP will automatically consider js/ts files with the same name as the service file.



   - Implement the assign and unassign actions.
     i. You can leverage req.subject as a shortcut for the information which entity is targeted by the bound action.
     ii. req.params contains the URL parameters like the ID of the bound entity.
     iii. Use CAPs SQL like query API to do the necessary statements.
     iv. Use req.info to let the student know that you assigned/unassigned them.
   - *(Optional):* You can also add service handler files for the other services and add input validations by using before UPDATE / SAVE event handlers.

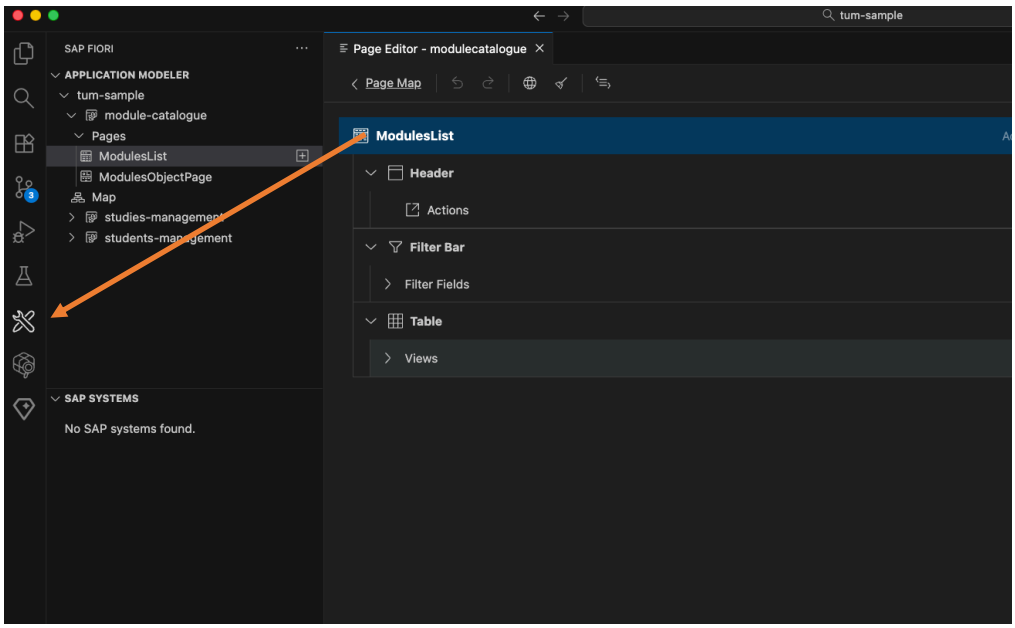
10. Create UI5 annotation, by using the Fiori Frontend Generator.
    (Note: You should repeat this step for each OData service, in total 3 times)
    - In the menu click **View->Command Palette (Ctrl + Shift + P)** and start typing Fiori and select ***Fiori: Open CF Application Generator***
    - Select ***List Report Page*** in the template section and click "Next"



11. In *Data Source* select "***Use a Local CAP Project***":
    - Choose a CAP Project: university_$$$###
    - OData Service: YourService
12. In the *Entity Selection* select:
    - Main Entity: YourMainEntityOfTheService
    - Navigation Entity: YourNavigationEntity
    - Automatically add table columns to the list page and a section to the object page if none already exists: No
    - Table Type: Responsive
13. In the *Project Attributes* select:
    - Module Name: e.g. students-management for the students-service
    - Application Title: e.g. Students Management for the students service
    - Application Namespace: sap.cap.sample
    - Description: YourDescription
    - Enable TypeScript: Yes
    - Add FLP Configuration: Yes
14. Continue creating the Fiori config for the remaining services, once you finished, continue with the next step.
    - The Semantic Object & Action is a unique combination in the Launchpads on SAP BTP and S4/HANA to navigate across apps by just specifying this combination instead of providing URLs, to make cross-app navigation

landscape agnostic. For the sample you can use any string value you like, the combinations should just be unique.

15. For defining the UI annotations you can use the Fiori application modeler plugin (icon):

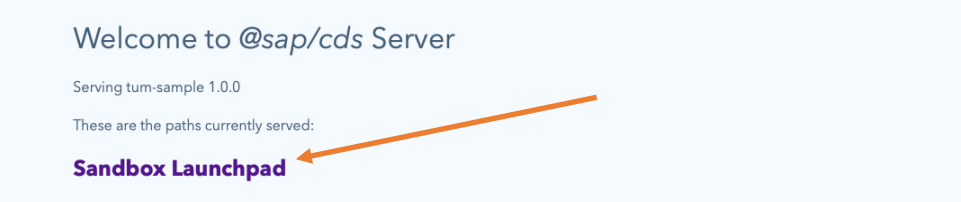


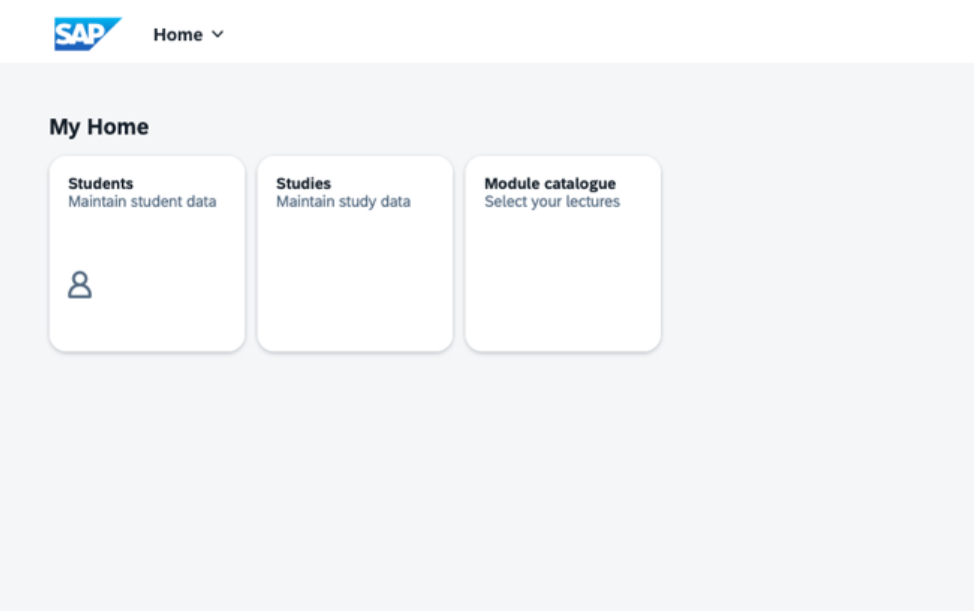
- Select the app you want to work on and select the page, e.g. either the list page or the details page.
- Afterwards you can configure columns for tables or modify the sections shown on the details page.
- You can also manually define the annotations in cds files. Ideally use the annotations.cds file in each app folder to do the annotations for the respective app.
    i. **@UI.LineItem** specifies a table definition
    ii. **@UI.HeaderInfo** specifies the header of an object page as well as the table name
    iii. **@UI.PresentationVariant** specifies the default sorting for a table
    iv. **@UI.FieldGroup** specifies a form
    v. **@UI.Facets** specifies the sections of an object page. Facets can contain field groups, line items or charts.
    vi. You can check-out the Fiori elemens showcase for samples of each annotation.
- To disable the “Go” button in the UI add the “liveMode” setting in each applications manifest.json file. For more information, reference to the docs.

16. Add a local launchpad by running the following command:

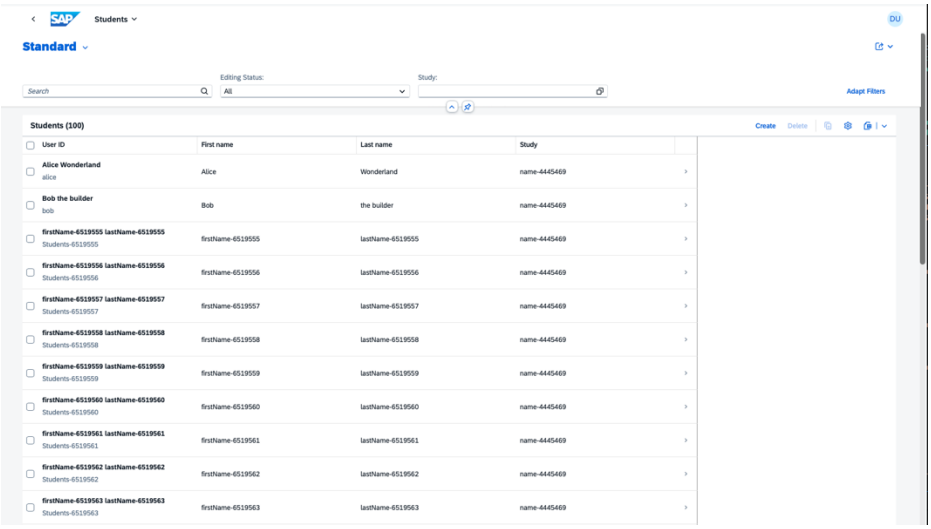
```
npm add -D cds-launchpad-plugin
```

Afterwards usually on localhost, port 4004 a ”Sandbox Launchpad” appears, via which you can test the UI apps within the SAP Shell. The Shell is important for features like cross-app navigation to work without issues.

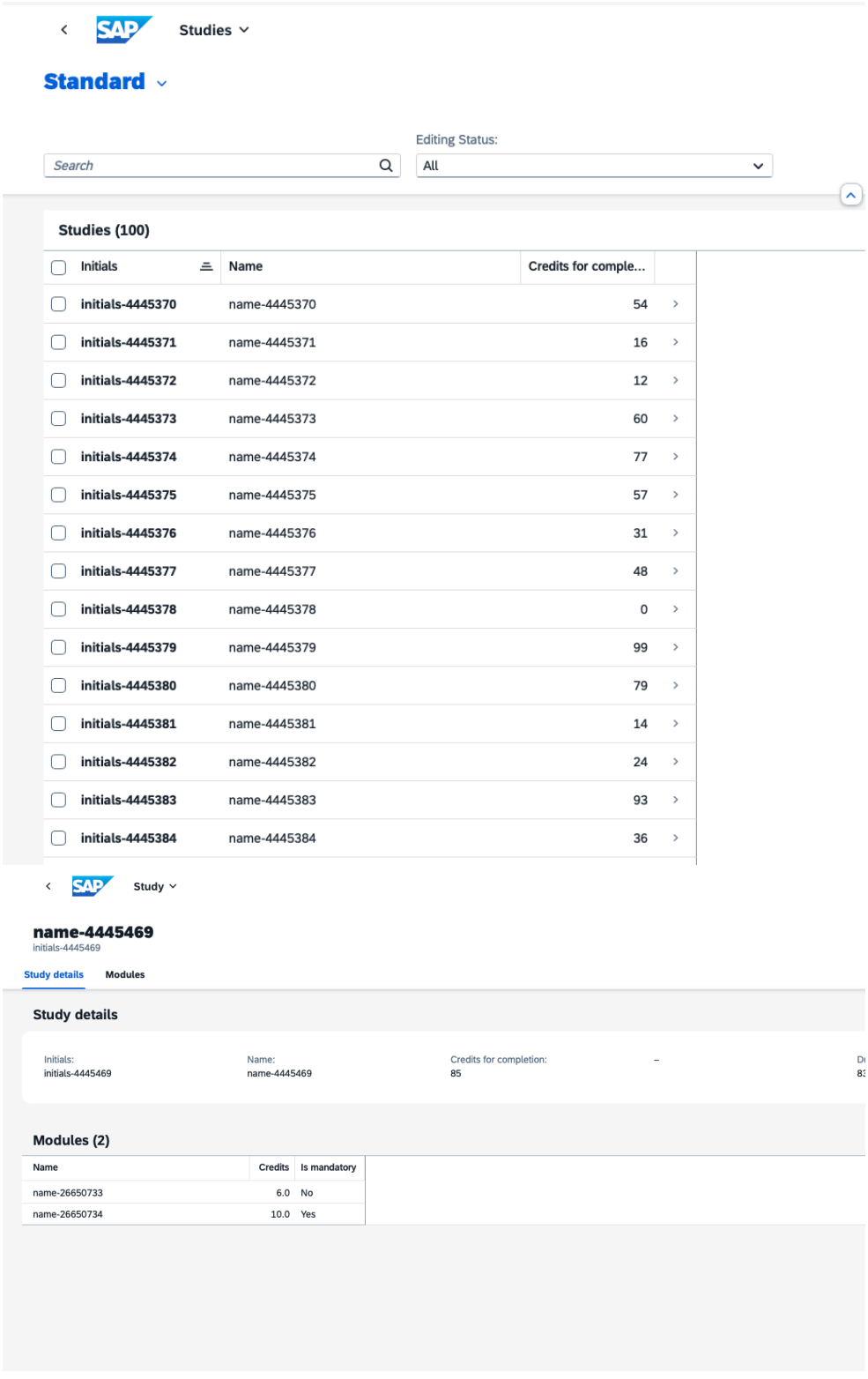




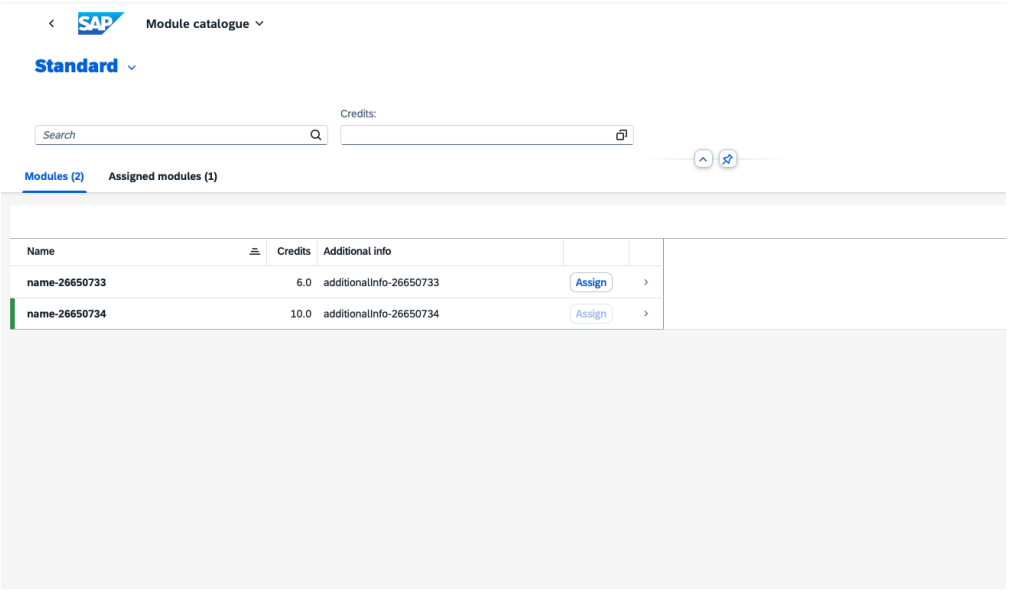
17. The UIs in the end could look like:
    - **Students** app:

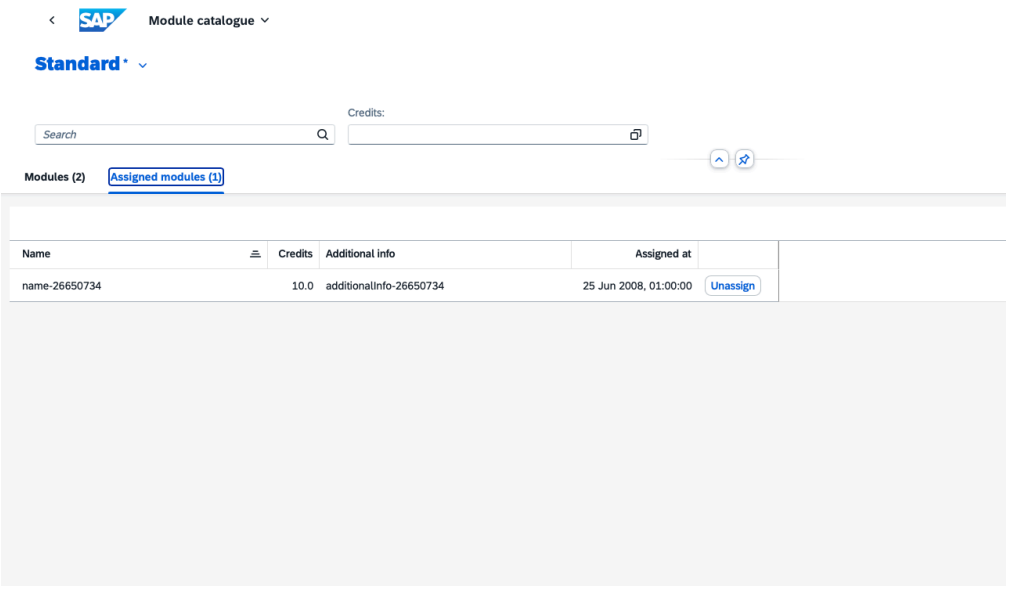


- **Studies** app:


Studies
Standard
Editing Status:
Search
All
Studies (100)
Initials
Name
Credits for comple...
initials-4445370 name-4445370 54
initials-4445371 name-4445371 16
initials-4445372 name-4445372 12
initials-4445373 name-4445373 60
initials-4445374 name-4445374 77
initials-4445375 name-4445375 57
initials-4445376 name-4445376 31
initials-4445377 name-4445377 48
initials-4445378 name-4445378 0
initials-4445379 name-4445379 99
initials-4445380 name-4445380 79
initials-4445381 name-4445381 14
initials-4445382 name-4445382 24
initials-4445383 name-4445383 93
initials-4445384 name-4445384 36
Study
name-4445469
initials-4445469
Study details
Modules
Study details
Initials:
initials-4445469
Name:
name-4445469
Credits for completion:
85
Modules (2)
Name
Credits
Is mandatory
name-26650733 6.0 No
name-26650734 10.0 Yes

- **Module catalogue** app:





- Use @Common.SideEffects to refresh parts of the UI when the actions are pressed.
- An example for having the actions as a table column is given in the Fiori Elements Showcase.
- Having multiple tabs can be configured via the manifest.json.
- Highlighting a row with color can be done via the so-called Criticality of the row: Sample but instead of a property reference you could use an expression:

```
isUserAssigned ? 3 : 0
```

  Note: number 3 is Positive and 0 corresponds to Neutral.
- Annotating @Core.OperationAvailable to the action itself in the service file allows you to disable the button on the UI. You can specify a property from the bound entity or an expression. Because `exists` is not possible in all annotation expressions, use a calculated field instead.

```
@Capabilities.Deletable : false
@Capabilities.Insertable : false
entity Modules as projection on persistence.Modules {
    *,
    (exists assignments[student.userID = $user.id] ? true : false) as isUserAssigned: Boolean @(UI.Hidden),
} actions {
    @(
        Core.OperationAvailable : (not $self.isUserAssigned),
        Common.SideEffects : {
            TargetProperties : ['in/isUserAssigned'],
            TargetEntities : ['/ModuleCatalogue.EntityContainer/ModuleAssignments']
        }
    )
    action assign();
};
```

(Optional) Deployment to CloudFoundry

18. Deploy the application
    a. Run:

```
cds add xsuaa,hana --for production
```

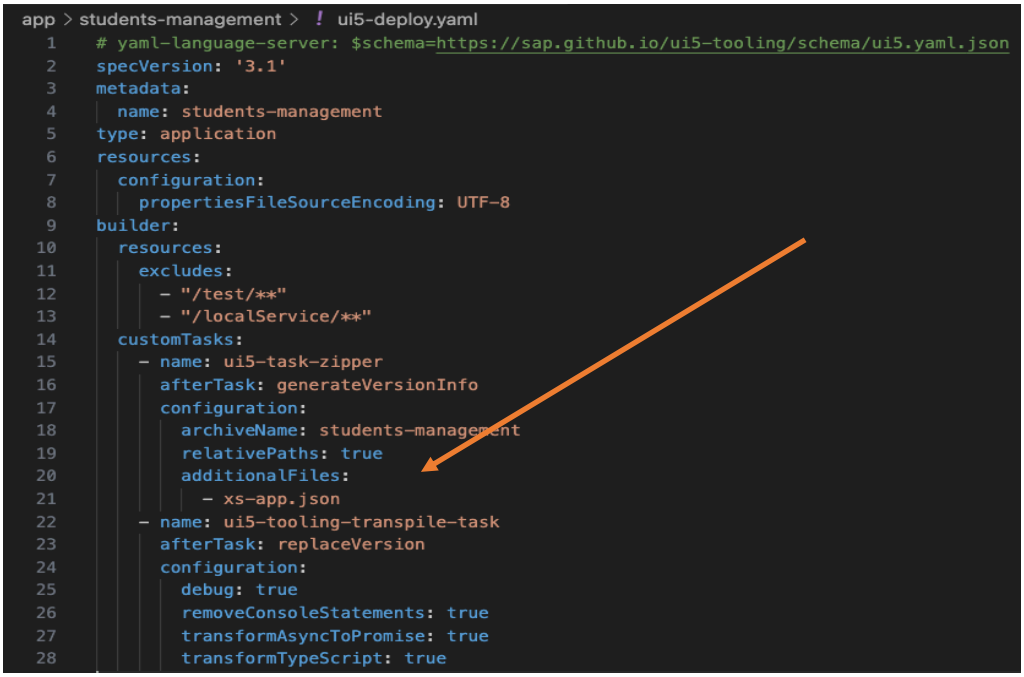
(This adds the necessary configuration for the authentication & authorization service (XSUAA) and for the HANA Cloud database.)

    b. Run:

```
cds add mta,workzone --for production
```

(This adds the necessary configuration for the deployment descriptor file (mta) and the UI deployment (Workzone is the central entry point for SAP solutions)

    c. **Please note:** Due to a current bug, please manually add "relativePaths: true" to every generated ui5-deploy.yaml file, like in the following image, line 19, and add the "ui5-tooling-transpile-task:":



    d. Run the following commands to install dependencies and deploy your app to CloudFoundry:

```
npm install
cds up
```

(*The command bundles the build command from mbt with the cf deploy command for deployment.)*

e. Due to the landscape limitations from the university, you can't access the apps from WorkZone but instead need to access them directly.
    i. For your project, you could follow the following tutorial with a BTP Trial account to deploy your app with a proper WorkZone integration.
    ii. For this lecture: Check your `mta.yaml` for a service name ending with `-destination`. Copy that name and run the following command:

    ```
    cf html5-list --di <destination-name> -u
    ```

    (this command will list the three HTML5 archives of your apps.)
    - In the URLs shown replace **cpp** with **launchpad** and then the app should open.
    - The Module catalogue app likely does not open as we defined role restrictions for it, this is due to a limitation on the university landscape.

f. For cleanup purposes or if you want to undeploy your application you can run the following command:

```
cf undeploy <ID from mta.yaml> --delete-services
```